

从技术上讲，普通迭代器与反向迭代器的关系反映了左闭合区间（参见 9.2.1 节，第 296 页）的特性。关键点在于 `[line.crbegin(), rcomma)` 和 `[rcomma.base(), line.cend())` 指向 `line` 中相同的元素范围。为了实现这一点，`rcomma` 和 `rcomma.base()` 必须生成相邻位置而不是相同位置，`crbegin()` 和 `cend()` 也是如此。



反向迭代器的目的是表示元素范围，而这些范围是不对称的，这导致一个重要的结果：当我们从一个普通迭代器初始化一个反向迭代器，或是给一个反向迭代器赋值时，结果迭代器与原迭代器指向的并不是相同的元素。

10.4.3 节练习

练习 10.34: 使用 `reverse_iterator` 逆序打印一个 `vector`。

练习 10.35: 使用普通迭代器逆序打印一个 `vector`。

练习 10.36: 使用 `find` 在一个 `int` 的 `list` 中查找最后一个值为 0 的元素。

练习 10.37: 给定一个包含 10 个元素的 `vector`，将位置 3 到 7 之间的元素按逆序拷贝到一个 `list` 中。

10.5 泛型算法结构



任何算法的最基本的特性是它要求其迭代器提供哪些操作。某些算法，如 `find`，只要求通过迭代器访问元素、递增迭代器以及比较两个迭代器是否相等这些能力。其他一些算法，如 `sort`，还要求读、写和随机访问元素的能力。算法所要求的迭代器操作可以分为 5 个迭代器类别（`iterator category`），如表 10.5 所示。每个算法都会对它的每个迭代器参数指明须提供哪类迭代器。

表 10.5: 迭代器类别		
输入迭代器	只读，不写；单遍扫描，只能递增	输入输出，单扫只增 可读可写，多扫只增 迭代器层次逐渐提高， 高层次的可以兼容低层次的
输出迭代器	只写，不读；单遍扫描，只能递增	
前向迭代器	可读写；多遍扫描，只能递增	
双向迭代器	可读写；多遍扫描，可递增递减	
随机访问迭代器	可读写，多遍扫描，支持全部迭代器运算	

第二种算法分类的方式（如我们在本章开始所做的）是按照是否读、写或是重排序列中的元素来分类。附录 A 按这种分类方法列出了所有算法。

算法还共享一组参数传递规范和一组命名规范，我们在介绍迭代器类别之后将介绍这些内容。

10.5.1 5 类迭代器



类似容器，迭代器也定义了一组公共操作。一些操作所有迭代器都支持，另外一些只有特定类别的迭代器才支持。例如，`ostream_iterator` 只支持递增、解引用和赋值。`vector`、`string` 和 `deque` 的迭代器除了这些操作外，还支持递减、关系和算术运算。

迭代器是按它们所提供的操作来分类的，而这种分类形成了一种层次。除了输出迭代器

器之外，一个高层类别的迭代器支持低层类别迭代器的所有操作。

411

C++标准指明了泛型和数值算法的每个迭代器参数的最小类别。例如，`find` 算法在一个序列上进行一遍扫描，对元素进行只读操作，因此至少需要输入迭代器。`replace` 函数需要一对迭代器，至少是前向迭代器。类似的，`replace_copy` 的前两个迭代器参数也要求至少是前向迭代器。其第三个迭代器表示目的位置，必须至少是输出迭代器。其他的例子类似。对每个迭代器参数来说，其能力必须与规定的最小类别至少相当。向算法传递一个能力更差的迭代器会产生错误。



对于向一个算法传递错误类别的迭代器的问题，很多编译器不会给出任何警告或提示。

迭代器类别

输入迭代器 (input iterator): 可以读取序列中的元素。一个输入迭代器必须支持

- 用于比较两个迭代器的相等和不相等运算符 (`==`、`!=`)
- 用于推进迭代器的前置和后置递增运算 (`++`)
- 用于读取元素的解引用运算符 (`*`)；解引用只会出现在赋值运算符的右侧
- 箭头运算符 (`->`)，等价于 `(*it).member`，即，解引用迭代器，并提取对象的成员

输入迭代器只用于顺序访问。对于一个输入迭代器，`*it++` 保证是有效的，但递增它可能导致所有其他指向流的迭代器失效。其结果就是，不能保证输入迭代器的状态可以保存下来并用来访问元素。因此，输入迭代器只能用于单遍扫描算法。算法 `find` 和 `accumulate` 要求输入迭代器；而 `istream_iterator` 是一种输入迭代器。

输出迭代器 (output iterator): 可以看作输入迭代器功能上的补集——只写而不读元素。输出迭代器必须支持

- 用于推进迭代器的前置和后置递增运算 (`++`)
- 解引用运算符 (`*`)，只出现在赋值运算符的左侧（向一个已经解引用的输出迭代器赋值，就是将值写入它所指向的元素）

我们只能向一个输出迭代器赋值一次。类似输入迭代器，输出迭代器只能用于单遍扫描算法。用作目的位置的迭代器通常都是输出迭代器。例如，`copy` 函数的第三个参数就是输出迭代器。`ostream_iterator` 类型也是输出迭代器。

前向迭代器 (forward iterator): 可以读写元素。这类迭代器只能在序列中沿一个方向移动。前向迭代器支持所有输入和输出迭代器的操作，而且可以多次读写同一个元素。因此，我们可以保存前向迭代器的状态，使用前向迭代器的算法可以对序列进行多遍扫描。算法 `replace` 要求前向迭代器，`forward_list` 上的迭代器是前向迭代器。

412

双向迭代器 (bidirectional iterator): 可以正向/反向读写序列中的元素。除了支持所有前向迭代器的操作之外，双向迭代器还支持前置和后置递减运算符 (`--`)。算法 `reverse` 要求双向迭代器，除了 `forward_list` 之外，其他标准库都提供符合双向迭代器要求的迭代器。

随机访问迭代器 (random-access iterator): 提供在常量时间内访问序列中任意元素的能力。此类迭代器支持双向迭代器的所有功能，此外还支持表 3.7（第 99 页）中的操作：

- 用于比较两个迭代器相对位置的关系运算符 (<、<=、>和>=)
- 迭代器和一个整数值的加减运算 (+、+=、-和-=)，计算结果是迭代器在序列中前进(或后退)给定整数个元素后的位置
- 用于两个迭代器上的减法运算符 (-)，得到两个迭代器的距离
- 下标运算符 (iter[n])，与*(iter[n])等价

算法 sort 要求随机访问迭代器。array、deque、string 和 vector 的迭代器都是随机访问迭代器，用于访问内置数组元素的指针也是。

10.5.1 节练习

练习 10.38: 列出 5 个迭代器类别，以及每类迭代器所支持的操作。

练习 10.39: list 上的迭代器属于哪类？vector 呢？

练习 10.40: 你认为 copy 要求哪类迭代器？reverse 和 unique 呢？

10.5.2 算法形参模式



在任何其他算法分类之上，还有一组参数规范。理解这些参数规范对学习新算法很有帮助——通过理解参数的含义，你可以将注意力集中在算法所做的操作上。大多数算法具有如下 4 种形式之一：**序列范围 + 【目标迭代器】 + 【其他参数】**

```
alg (beg, end, other args);
alg (beg, end, dest, other args);
alg (beg, end, beg2, other args);
alg (beg, end, beg2, end2, other args);
```

其中 alg 是算法的名字，beg 和 end 表示算法所操作的输入范围。几乎所有算法都接受一个输入范围，是否有其他参数依赖于要执行的操作。这里列出了常见的一种——dest、beg2 和 end2，都是迭代器参数。顾名思义，如果用到了这些迭代器参数，它们分别承担指定目的位置和第二个范围的角色。除了这些迭代器参数，一些算法还接受额外的、非迭代器的特定参数。

413

接受单个目标迭代器的算法

dest 参数是一个表示算法可以写入的目的位置的迭代器。算法假定 (assume): 按其需要写入数据，不管写入多少个元素都是安全的。



WARNING

向输出迭代器写入数据的算法都假定目标空间足够容纳写入的数据。

dest 常见的使用方法

如果 dest 是一个直接指向容器的迭代器，那么算法将输出数据写到容器中已存在的元素内。更常见的情况是，dest 被绑定到一个插入迭代器 (参见 10.4.1 节，第 358 页) 或是一个 ostream_iterator (参见 10.4.2 节，第 359 页)。插入迭代器会将新元素添加到容器中，因而保证空间是足够的。ostream_iterator 会将数据写入到一个输出流，同样不管要写入多少个元素都没有问题。

接受第二个输入序列的算法

接受单独的 `beg2` 或是接受 `beg2` 和 `end2` 的算法用这些迭代器表示第二个输入范围。这些算法通常使用第二个范围中的元素与第一个输入范围结合来进行一些运算。

如果一个算法接受 `beg2` 和 `end2`，这两个迭代器表示第二个范围。这类算法接受两个完整指定的范围：`[beg, end)` 表示的范围和 `[beg2, end2)` 表示的第二个范围。

只接受单独的 `beg2` (不接受 `end2`) 的算法将 `beg2` 作为第二个输入范围中的首元素。此范围的结束位置未指定，这些算法假定从 `beg2` 开始的范围与 `beg` 和 `end` 所表示的范围至少一样大。



接受单独 `beg2` 的算法假定从 `beg2` 开始的序列与 `beg` 和 `end` 所表示的范围至少一样大。



10.5.3 算法命名规范

除了参数规范，算法还遵循一套命名和重载规范。这些规范处理诸如：如何提供一个操作代替默认的 `<或==` 运算符以及算法是将输出数据写入输入序列还是一个分离的目的位置等问题。

一些算法使用重载形式传递一个谓词

接受谓词参数来代替 `<或==` 运算符的算法，以及那些不接受额外参数的算法，通常都是重载的函数。函数的一个版本用元素类型的运算符来比较元素；另一个版本接受一个额外谓词参数，来代替 `<或==`：

```
unique(beg, end);           // 使用 == 运算符比较元素
unique(beg, end, comp);     // 使用 comp 比较元素
```

两个调用都重新整理给定序列，将相邻的重复元素删除。第一个调用使用元素类型的 `==` 运算符来检查重复元素；第二个则调用 `comp` 来确定两个元素是否相等。由于两个版本的函数在参数个数上不相等，因此具体应该调用哪个版本不会产生歧义（参见 6.4 节，第 208 页）。

if 版本的算法

接受一个元素值的算法通常有另一个不同名的（不是重载的）版本，该版本接受一个谓词（参见 10.3.1 节，第 344 页）代替元素值。接受谓词参数的算法都有附加的 `if` 前缀：

```
find(beg, end, val);        // 查找输入范围中 val 第一次出现的位置
find_if(beg, end, pred);    // 查找第一个令 pred 为真的元素
```

这两个算法都在输入范围中查找特定元素第一次出现的位置。算法 `find` 查找一个指定值；算法 `find_if` 查找使得 `pred` 返回非零值的元素。

这两个算法提供了命名上差异的版本，而非重载版本，因为两个版本的算法都接受相同数目的参数。因此可能产生重载歧义，虽然很罕见，但为了避免任何可能的歧义，标准库选择提供不同名字的版本而不是重载。

区分拷贝元素的版本和不拷贝的版本

默认情况下，重排元素的算法将重排后的元素写回给定的输入序列中。这些算法还提供另一个版本，将元素写到一个指定的输出目的位置。如我们所见，写到额外目的空间的

算法都在名字后面附加一个 `_copy`（参见 10.2.2 节，第 341 页）：

```
reverse(beg, end);           // 反转输入范围中元素的顺序
reverse_copy(beg, end, dest); // 将元素按逆序拷贝到 dest
```

一些算法同时提供 `_copy` 和 `_if` 版本。这些版本接受一个目的位置迭代器和一个谓词：

```
// 从 v1 中删除奇数元素
remove_if(v1.begin(), v1.end(),
          [](int i) { return i % 2; });
// 将偶数元素从 v1 拷贝到 v2; v1 不变
remove_copy_if(v1.begin(), v1.end(), back_inserter(v2),
               [](int i) { return i % 2; });
```

两个算法都调用了 `lambda`（参见 10.3.2 节，第 346 页）来确定元素是否为奇数。在第一个调用中，我们从输入序列中将奇数元素删除。在第二个调用中，我们将非奇数（亦即偶数）元素从输入范围拷贝到 `v2` 中。

10.5.3 节练习

415

练习 10.41: 仅根据算法和参数的名字，描述下面每个标准库算法执行什么操作：

```
replace(beg, end, old_val, new_val);
replace_if(beg, end, pred, new_val);
replace_copy(beg, end, dest, old_val, new_val);
replace_copy_if(beg, end, dest, pred, new_val);
```

10.6 特定容器算法

与其他容器不同，链表类型 `list` 和 `forward_list` 定义了几个成员函数形式的算法，如表 10.6 所示。特别是，它们定义了独有的 `sort`、`merge`、`remove`、`reverse` 和 `unique`。通用版本的 `sort` 要求随机访问迭代器，因此不能用于 `list` 和 `forward_list`，因为这两个类型分别提供双向迭代器和前向迭代器。

链表类型定义的其他算法的通用版本可以用于链表，但代价太高。这些算法需要交换输入序列中的元素。一个链表可以通过改变元素间的链接而不是真的交换它们的值来快速“交换”元素。因此，这些链表版本的算法的性能比对应的通用版本好得多。

Best Practices

对于 `list` 和 `forward_list`，应该优先使用成员函数版本的算法而不是通用算法。

表 10.6: <code>list</code> 和 <code>forward_list</code> 成员函数版本的算法	
这些操作都返回 <code>void</code>	
<code>lst.merge(lst2)</code>	将来自 <code>lst2</code> 的元素合并入 <code>lst</code> 。 <u><code>lst</code> 和 <code>lst2</code> 都必须是有顺序的。</u>
<code>lst.merge(lst2, comp)</code>	元素将从 <code>lst2</code> 中删除。在合并之后， <code>lst2</code> 变为空。第一个版本使用 <code><</code> 运算符；第二个版本使用给定的比较操作
<code>lst.remove(val)</code>	调用 <code>erase</code> 删除掉与给定值相等 (<code>==</code>) 或令一元谓词为真的每个元素
<code>lst.remove_if(pred)</code>	
<code>lst.reverse()</code>	反转 <code>lst</code> 中元素的顺序